

PROJECT REPORT

Multi-Level Cache Architecture Simulator

L1/L2 Cache Hierarchy — C++17 + Python Analytics Pipeline

Submitted by

Priya Kumari Shah

Language: C++17 • Build: CMake • Analytics: Python + matplotlib • June 2026

1. Problem Statement

Modern CPUs are orders of magnitude faster than DRAM. A typical desktop core can issue an instruction every nanosecond; a main-memory access takes 50–100 ns. Without a cache hierarchy, the processor would stall on virtually every memory access and utilise only a fraction of its compute throughput.

Designing a cache hierarchy — choosing capacity, associativity, block size, write policy, and eviction policy — is non-trivial. The interactions between these parameters are non-linear: doubling associativity can eliminate nearly all conflict misses at one capacity but deliver diminishing returns at another.

The specific problems this project addresses:

- Simulate a two-level (L1 + L2) cache hierarchy with bit-accurate address decomposition
- Support two write policies (Write-Back/Write-Allocate and Write-Through/No-Write-Allocate) with correct dirty-block propagation
- Support two eviction policies (LRU and FIFO) within a set-associative structure
- Enforce inclusive coherence: when L2 evicts a block, L1 must be invalidated
- Identify and eliminate a subtle double-eviction bug in miss-fill logic
- Automate parameter sweeping across 45+ configurations and generate quantitative performance charts

2. End Goal

Deliver a fully configurable cache simulator that a hardware architect or student can use to:

- Run any industry-standard memory trace (Valgrind, Dinero format) and obtain L1 and L2 hit/miss statistics
- Change every architectural parameter at runtime via CLI — no recompilation
- Compare write policies and eviction policies against the same workload with a single command
- Generate publication-ready charts showing how hit rate changes with associativity, cache size, and eviction policy
- Extend the codebase cleanly — new policies slot in without modifying existing call sites

3. System Architecture

The simulator is built in three layers, each with a single clear responsibility:

Layer	File(s)	Responsibility
CacheLine	CacheLine.h	Per-line state: valid/dirty flags, tag, LRU and FIFO counters.

CacheSet	CacheSet.h / .cpp	Set-associative slot management: hit probe, eviction selection, block install, invalidation.
Cache	Cache.h / .cpp	Address decomposition, hierarchy cascade, write-policy enforcement, inclusive invalidation, statistics.
main.cpp	main.cpp	CLI parsing, trace-file loop, result printing.
cache_sweeper.py	Scripts/	Automated parameter sweep, subprocess invocation, matplotlib chart generation, CSV export.

The Cache objects are wired at runtime: `l1.set_next_level(&l2)`. A miss at L1 triggers a read from L2, which may in turn go to main memory. No pointers are owned by the hierarchy — callers manage lifetimes, keeping the design composable for future three-level hierarchies.

4. Steps Involved

Step 1 — Address Decomposition Design

Before any data structure could be designed, the way a 64-bit address maps to a cache set had to be defined. Two approaches exist:

Approach	Implementation	Verdict
String parsing	Split hex string, compute modulo	Fragile, slow, error-prone for edge addresses
Bitwise shifts and masks ✓	<code>index = (addr >> offset_bits) & index_mask; tag = addr >> (offset+index bits)</code>	Correct for all 64-bit addresses, ~10x faster, matches hardware exactly

The bitwise approach was chosen unconditionally. The bit widths are computed once at construction from the configured `block_size` and `num_sets` (both required to be powers of two, validated in the constructor). Formula: `offset_bits = log2(block_size)`, `index_bits = log2(num_sets)`. All remaining high-order bits form the tag.

Step 2 — Cache Line and Set Design

The `CacheLine` struct holds exactly the state a real cache line carries: a valid bit, a dirty bit, the tag, and two counters (one for LRU, one for FIFO). Both counters are maintained simultaneously so the eviction policy can be switched without structural changes.

The `CacheSet` `access()` method walks through four states in order:

- **Hit probe:** scan all ways; if tag matches a valid line, mark dirty if write, update LRU counter, return hit
- **No-write-allocate shortcut:** if this is a write miss and `allocate=false`, return immediately without touching any line

- **Empty slot:** if any way is invalid, install the new block there without eviction
- **Eviction:** call `find_victim()` (pure query — reads counters but modifies nothing), capture dirty/tag metadata into `AccessResult`, overwrite victim in-place

Step 3 — Identifying and Fixing the Double-Eviction Bug

This is the most significant engineering contribution of the project.

The Bug

Earlier implementations of `Cache::handle_miss()` called `sets[index].access()` twice: first to detect the miss, then again to install the newly fetched block. The second call re-ran the eviction logic on the same set, which could select a different victim than the first call (because `access_clock_` had advanced) and evict a block that had just been legitimately installed. This is a silent correctness bug — no crash, but dirty data silently discarded.

The Fix

Miss handling was decomposed into distinct phases:

- **Phase 1 (fetch):** call `next_level_>access('R', address)` to bring the block into L2 if absent
- **Phase 2 (install):** call `sets[index].access(tag, is_write, evpol, allocate=true)` — guaranteed to perform exactly one eviction, not a spurious second one
- **Phase 3 (writeback):** if `AccessResult::writeback_needed` is true, reconstruct the evicted address and write it back to L2

The `AccessResult` struct was introduced specifically to carry eviction metadata without requiring output parameters or a second access call. A future Pseudo-LRU policy can attach additional metadata to `AccessResult` without changing any caller.

Step 4 — Write Policy Implementation

Policy	On Write Hit	On Write Miss	Typical Use
WBWA ✓ (default)	Mark line dirty — write to lower level deferred until eviction	Fetch block, install, mark dirty	L1 and L2 in most CPUs
WTWNA	Write through to lower level immediately	Forward write to lower level, do NOT install block	Simple L1 or LLC-to-DRAM path

WBWA was chosen as the default because it minimises write traffic to L2. WTWNA is provided for completeness and for modelling write-intensive workloads.

Step 5 — Inclusive Coherence Protocol

A strict inclusive L2 means every block in L1 must also be present in L2. When L2 evicts a block, L1 must invalidate it — otherwise L1 holds stale data.

- When L2 evicts a dirty block under WBWA, `Cache::handle_miss()` calls `next_level_>invalidate_block(wb_addr)` after writing back to L2

- `invalidate_block()` reconstructs the set index from the block address and calls `CacheSet::invalidate(tag, was_dirty)`
- `probe()` is provided as a non-modifying presence check for testing

The ordering is critical: L1 writeback to L2 must complete before L1 invalidates, ensuring coherence order matches real hardware.

Step 6 — Eviction Policy Implementation

Policy	Counter Used	Updated When	Trade-off
LRU ✓ (default)	<code>lru_counter</code>	On every cache hit	Tracks recency accurately; better hit rate for temporal locality workloads
FIFO	<code>fifo_counter</code>	Only on first load (block install)	Simpler hardware; does not punish frequently-accessed old blocks

Both counters are stored on every CacheLine and maintained in parallel. Switching between LRU and FIFO costs nothing at runtime — just a ternary on the counter field in `find_victim()`.

Step 7 — CLI and Trace File Parsing

The entire L1 and L2 configuration is passed via CLI arguments (defaulting to 16KB/256KB LRU WBWA). This lets the Python sweeper invoke the binary hundreds of times without recompilation. The trace parser handles R, W, and I (instruction fetch, treated as read) operations, skips comment and blank lines, and emits a warning (not a crash) for malformed address fields.

Step 8 — Python Analytics Pipeline

The `cache_sweeper.py` script drives the simulator as a subprocess, parses its stdout with regex, and runs three sweeps:

- **Associativity sweep:** 1-way through 16-way, fixed L1=4KB, L2=256KB, LRU, WBWA
- **Cache size sweep:** L1 from 1KB to 32KB (sets from 16 to 512), fixed 4-way, LRU, WBWA
- **Eviction policy comparison:** LRU vs FIFO across 1-way through 16-way

Results are saved as both CSV (for further analysis) and matplotlib PNG charts. The pipeline is designed to be re-run on any trace file, making it reusable for regression benchmarking.

5. What Was Built

5.1 CacheLine — Per-Line State

A struct with five fields: `valid (bool)`, `dirty (bool)`, `tag (uint64_t)`, `lru_counter (uint32_t)`, `fifo_counter (uint32_t)`. Header-only — no `.cpp` needed. Two separate counters allow both LRU and FIFO to be evaluated on the same dataset without any conversion.

5.2 CacheSet — Set-Associative Slot Manager

Holds a `std::vector<CacheLine>` of size = associativity. The core `access()` method implements the four-phase state machine. `find_victim()` is a pure const query — it reads counters but does not modify any line, ensuring eviction selection and block installation are always separate operations.

Method	Complexity	Notes
<code>access(tag, is_write, policy, allocate)</code>	$O(\text{ways})$	Linear scan; ways is 1–16 in practice, effectively $O(1)$
<code>invalidate(tag, was_dirty)</code>	$O(\text{ways})$	Called by inclusive protocol on L2 eviction
<code>contains(tag)</code>	$O(\text{ways})$	Non-modifying presence check
<code>find_victim(policy)</code>	$O(\text{ways})$	Pure query; never modifies state

5.3 Cache — Hierarchy Controller

- **`access(type, address)`**: parse address, probe set, handle write-through on hit, call `handle_miss()` on miss
- **`handle_miss()`**: fetch from next level, install into set, write back evicted dirty block, invalidate in L1 if inclusive
- **`invalidate_block(address)`**: parse address, call `set.invalidate()`, used by inclusive protocol
- **`reconstruct_address(tag, index)`**: reverse the address decomposition — builds writeback address from evicted tag + set index
- **`print_stats()`**: formatted output with total accesses, read/write hit/miss counts, writeback count, and percentages

5.4 Empirical Results from the Sweep Pipeline

Associativity vs Hit Rate (L1=4KB, L2=256KB, LRU, WBWA)

L1 Associativity	L1 Hit Rate	L1 Miss Rate	L2 Hit Rate	L2 Miss Rate
1-way (direct-mapped)	58.28%	41.72%	94.39%	5.61%
2-way	73.78%	26.22%	91.08%	8.92%
4-way	85.30%	14.70%	84.08%	15.92%
8-way	92.71%	7.29%	67.91%	32.09%
16-way	96.91%	3.09%	24.17%	75.83%

Moving from 1-way to 4-way eliminates 65% of L1 misses. Beyond 8-way, gains diminish sharply — the workload's conflict miss rate is essentially zero at that point. Higher L1 associativity also reduces L2 traffic, which explains the falling L2 hit rate (L2 sees fewer requests, so the hit rate denominator shrinks).

Cache Size vs Hit Rate (4-way, LRU, WBWA)

L1 Size	L1 Sets	L1 Hit Rate	L1 Miss Rate	L2 Hit Rate	L2 Miss Rate
1 KB	16	66.26%	33.74%	93.06%	6.94%
2 KB	32	75.36%	24.64%	90.50%	9.50%
4 KB	64	85.30%	14.70%	84.08%	15.92%
8 KB	128	92.18%	7.82%	70.09%	29.91%
16 KB	256	96.74%	3.26%	28.22%	71.78%
32 KB	512	97.66%	2.34%	0.00%	100.00%

At 32 KB, L1 absorbs virtually the entire working set — L2 hit rate drops to 0% because L1 never misses. This confirms the benchmark trace has a working set of roughly 16–32 KB.

6. Scalability Analysis

6.1 Simulation Performance

The simulator processes memory accesses at over 1 million accesses per second on a modern desktop CPU. This is sufficient to simulate hours of real program execution in seconds.

Dimension	Current Behaviour	Limit
Addresses per second	> 1M/sec (Release build)	Bounded by branch prediction in way scan; effectively unlimited for benchmarking
Ways (associativity)	1 to any power of 2	Linear in ways — 64-way adds ~16ns/access; hardware never exceeds 32-way
Number of sets	Any power of 2 (validated)	Memory: sets × ways × sizeof(CacheLine) ≈ sets × ways × 20 bytes
Hierarchy depth	Currently L1 + L2	Trivially extensible: add Cache I3; <code>l2.set_next_level(&l3)</code>
Trace file size	Streams line-by-line	No limit — trace is never loaded fully into memory

6.2 Extending to More Cache Levels

The cascade design means adding an L3 is a two-line change in `main.cpp`: instantiate `Cache I3(I3_cfg)` and call `l2.set_next_level(&l3)`. No changes to `CacheSet`, `CacheLine`, or any existing Cache logic are needed. The inclusive protocol propagates through the chain automatically.

6.3 Parallel Sweep Throughput

The Python sweeper currently runs each simulator invocation serially. For large parameter grids (e.g., 5 associativity × 6 sizes × 2 policies × 2 write policies = 120 configurations), total sweep time on the benchmark trace is approximately 2 minutes.

If sweep time becomes a bottleneck, the sweeper can be parallelised with Python's `concurrent.futures.ProcessPoolExecutor` in ~10 lines — the simulator binary is stateless and safe to run concurrently. With 8 parallel workers, sweep time would drop to ~15 seconds.

6.4 Production Deployment Considerations

Scale	Architecture	Estimated Cost (AWS)
Single user / local	Binary on developer machine	\$0 — runs locally
Research group (10–50 concurrent sweeps)	t3.xlarge (4 vCPU) with parallel subprocess pool	~\$120/month
SaaS tool (thousands of sweeps/day)	Containerised binary on ECS Fargate, S3 for charts, API Gateway front-end	~\$300–600/month depending on trace sizes
Large-scale batch (millions of trace replays)	AWS Batch with spot instances, S3 input/output, DynamoDB for result metadata	~\$0.30–0.50 per 1000 simulation runs (spot pricing)

7. Challenges Faced & Learnings

7.1 Challenges

- **The double-eviction bug:** The bug was completely silent — no crash, no assertion failure, just slightly wrong hit rate numbers. Catching it required writing a test that compared the simulator output against a hand-traced execution on a small, known input. Without that test, the bug would have produced plausible-but-incorrect statistics.
- **Inclusive coherence ordering:** Getting the ordering right — L1 writeback to L2 must happen before L1 invalidates the line — required careful reasoning about what 'consistent' means when two levels can hold the same block. An incorrect ordering can produce a window where neither level holds the valid data.
- **FIFO counter reset on eviction:** When a new block was installed into an evicted slot, it initially inherited the victim's `fifo_counter`, corrupting birth-time tracking. This made FIFO behave unpredictably at high associativity. The fix was to reset both `lru_counter` and `fifo_counter` to `access_clock_` on every block install.
- **WTWNA write-miss bypass:** Write misses under WTWNA were initially being allocated (block installed in cache) instead of bypassed. This is functionally incorrect and wastes cache capacity on data that won't be re-read. Adding the early-return path in `access()` for `is_write && WTWNA` fixed this.

7.2 Key Learnings

- **Silent bugs are the most dangerous:** All four major bugs identified produced plausible-looking output. None crashed. The lesson is that correctness tests must check invariants,

not just that the program runs — verify that the hit rate for a known trace matches the hand-calculated expected value.

- **Separation of query and mutation:** Making `find_victim()` a pure const query that never modifies state was the key design decision that prevented the double-eviction bug from recurring. Any method that both selects and applies an eviction is inherently fragile.
- **Parameter interactions are non-linear:** The empirical data confirmed that L1 associativity and L2 hit rate move in opposite directions — a result that isn't obvious from first principles but emerges clearly from the sweep. Running the simulator was more informative than reasoning from theory alone.
- **CLI-driven configuration unlocks automation:** Designing the binary to accept all parameters via CLI rather than compile-time constants was a deliberate choice that paid off immediately — the Python sweeper was trivial to write because each configuration was just a different subprocess invocation.

8. Key Bug Fixes Summary

Issue	Root Cause	Fix Applied
Double-eviction bug	<code>access()</code> called twice on miss — second call triggered a second eviction.	Separated fetch from install. Introduced <code>AccessResult</code> struct.
Inclusive coherence missing	L2 eviction did not signal L1 invalidation.	Added <code>next_level_>invalidate_block(wb_addr)</code> after writeback in <code>handle_miss()</code> .
WTWNA write-miss bypass missing	Write misses under WTWNA were being allocated instead of bypassed.	Added early return in <code>access()</code> for <code>is_write && WTWNA</code> before calling <code>handle_miss()</code> .
FIFO counter reset on eviction	New block inherited victim's <code>fifo_counter</code> , corrupting birth-time tracking.	Reset both <code>lru_counter</code> and <code>fifo_counter</code> to <code>access_clock_</code> on block install.

9. Known Limitations & Next Steps

Limitation	Recommended Next Step
No L3 cache	Add Cache I3 instance and <code>l2.set_next_level(&l3)</code> in <code>main.cpp</code> . No other changes needed.
Eviction policy string comparison inside hot loop	Change <code>eviction_policy</code> from <code>std::string</code> to <code>EvictionPolicy</code> enum. Eliminates string comparison overhead.
No Pseudo-LRU or Random replacement	Extend <code>find_victim()</code> with additional policy variants. <code>AccessResult</code> already supports future metadata.

Python sweeper is single-threaded	Replace serial loop with <code>concurrent.futures.ProcessPoolExecutor</code> for ~8x throughput improvement.
No prefetching model	Add a stride-prefetcher layer that intercepts <code>access()</code> calls and pre-loads adjacent blocks.

10. Summary

This project delivers a functionally correct, fully configurable L1/L2 cache hierarchy simulator. Every major architectural decision — bitwise address decomposition, the `AccessResult` struct, two write policies, two eviction policies, inclusive coherence, and the decoupled Python analytics pipeline — was made after evaluating concrete alternatives.

The most significant contribution is identifying and fixing the double-eviction bug. The buggy implementation produced incorrect statistics silently — hit rates appeared plausible but were systematically wrong. The fix separates fetch from install and introduces `AccessResult`, not only correcting the bug but producing a cleaner API that makes future policy extensions easier.

The empirical sweep data confirms the theoretical predictions: conflict misses dominate at low associativity, capacity misses dominate as the working set exceeds cache size, and LRU outperforms FIFO at high associativity due to temporal locality exploitation. These findings are quantified with reproducible data rather than asserted from textbook knowledge.

Priya Kumari Shah

June 2026